Art analysis to import publications

This has been written in 2025.

The main objective is to retrieve some information about a publication and then to store it in the *R3MOB SQL* database.

Where to search - Academic Databases

There are a lot of databases around the world that store scientific publications.

Among themselves, there are for instance:

- ***www.science.gov***: a *U.S government* database that contains scientific research from *U.S federal agencies*.

This one contains more than *200 million pages* of scientific information for free, and is a gateway to over 2000 scientific websites. However, it is centered on the *U.S*.

- ***https://csxstatic.ist.psu.edu/home*** (*CiteSeer*): centered on computer and information science.
- ***https://ieeexplore.ieee.org/Xplore/home.jsp*** (*IEEE Xplore*): very famous, however, it requires a subscription.
- ***https://www.springer.com*** (*Springer*): historical database, and also requires a subscription.

Anyway, you can follow **this link** to get a full list of academic databases.

Among those databases, there are the one which are:

- **Not Free**: import a publication from it will be hard.
- **Free**: import a publication from it will be possible.

So the idea is to search for the publication in the right database. The main idea is to find something that can make request to all those databases.

How to search - Search Engine

The databases discussed above also contain a *search engine*. From the user perspective, it is a *research bar*.

Indexer and Search Library

The *Search library* is the *core* of the search engine. It contains all the algorithmic stuff to be as efficient and accurate as possible.

For instance, let's talk about ***Apache Lucene***. This is an *open source Java* library providing indexing and search features, such as spellchecking, highlighting and advanced tokenization capabilities.

It is used by websites like *Twitter*, *Apple*, and even *Wikipedia*.

However, *Lucene* does not contain *crawling* and *HTML parsing* functionalities. That's why it is not recommended to use it directly.

Lucene-based Search Engines

Many services already extended *Lucene*'s capabilities of parsing, such as:

- ***Apache Nutch***: a *web crawler*, It provides *web crawling* and *HTML* parsing. **Wikipedia** says:

  > Web crawler, sometimes called a spider or spiderbot and often shortened to crawler, is an Internet bot that systematically browses the World Wide Web and that is typically operated by search engines for the purpose of Web indexing.

For instance, the *GoogleBot* is *Google*'s generic web crawler that is responsible for crawling sites that will show up on *Google*'s search engine, ***Kinsta.com*** says.

Another example could be **citeseerxbot**, the crawler of ***CiteSeerX***, discussed above.

- ***Apache Solr***: an 2004 *enterprise-search* platform. It can also *crawl* some web content as well as public databases. This is exactly what it is needed to search for publications.

It is used by ***Apache Hadoop*** which is a very famous framework for *distributed computing*.

- ***ElasticSearch***: also an *enterprise-search* platform, but this one has been released in 2010. It is currently the most popular *search engine*, even if it seems that it is only working using *json* format.

Differences between Solr and ElasticSearch

Both **ElasticSearch** and **Apache Solr** are very powerful. However, the *ElasticSearch API* is only working using *json* format.

ElasticSearch is actually replacing every *Solr*-based app thanks to its ability to integrate *NLP* (Natural Language Processing) and, for instance, **BERT**.

Here is ***a more detailed explanation***.

Which tools are designed to search specifically publications

The *search engines* discussed above are already great to use. They give access to a *REST Api*, with nice documentation. However, they are too generic, they are not configured for especially *scientific publications*.

That's why I'll introduce here mostly two services that are based on a *search engine* and that can be used to search for publications among public databases.

1. ***Crossref***: A *RESTFUL Api* that has been recently updated to use *ElasticSearch* instead of *Solr*. That's why some of the content is a little bit deprecated, but it seems to be the best option for this project.

It is specialized on *research objects*. You can **try it here**.

It is so impressive because *ElasticSearch* is based on a *Peer 2 Peer* architecture. *Crossref* gathers more than *22,000* members across the world, with nearly *2 billion monthly API queries*.

One difficulty could be to *construct the DOIs* of a broken publication if the user wants to import a publication with, actually, a broken *DOI*. Indeed, the *DOIs* (example: *10.1037/0003-066x.59.1.29*) could break a link, because of their format. So this type of error will be hard to solve, a publication may not be retrieved if the user gives a *DOI*.

2. ***Semantic Scholar***: A *REST Api*. Its corpus gathers *214 Million* papers and more than *79 Million* authors.

It is possible to get an *Api key* **right here**. Without an *Api key*, the limitation is set to *100* requests every 5minutes. However, *Semantic Scholar* does not *crawl/index* for material that is behind a *paywall*, **wikipedia** says.

This one is very powerful because it is natively based on *NLP*. The main advantadge compared to *Crossref* is its capability to analyse the content.

You can try it **right here**.

My plan to import publications on the website

There are some publications that are not available freely on the internet. One may take the example of *IEEE Xplore* database, even the *abstract* is private. To then classify the publication rightfully, the abstract is at least needed.

The user may want to import a publication from *IEEE* database, or may not. That's why the user needs to be able to import the publication using a file.

Import using a file

The idea is to build a *Javascript* module that will:

1. parse the given file, in *json* format, because the former dev on the site chose *json* as main file type, and because *ElasticSearch*, so *Crossref*, only gives *json* files.

The different file formats could all be parsed in *json* using external modules such as for instance:

- **Bibtex JS** to parse *Bibtex* files.

- **XML JS** to parse *XML* files.
- etc..

2. put the data in this *json* file, into the *MySQL* database, using, as the former dev used before me, **Sequelize**.

I assume that all this part of parsing should be done on the *client side*.

Import using a search engine

The idea is to use the *Crossref API*, to retrieve *json* files and then put it in the *MySQL* database using *Sequelize*.

I assume that all this part of parsing should be done on the *client side*, to retrieve the publication.

However, when the publication is retrieved, one important thing may be to also find the authors. So here the *Semantic Scholar API* could be used to retrieve data about the authors because this *API* has access to more than *79 Million* authors.

I assume that all this searching part should be done on the *server side*.

Besides, it could be possible to retrieve very cool analytics from *Semantic Scholar API*, such as **themes**, and even **AI-generated topics and sub-themes**.

How to use the Crossref tool

There are multiple implementations that simplify the use of *Crossref API*. Here are some examples from ***this website***:

1. ***Crossref Commons***: it is an official client from *Crossref*, written in *Python*. However, it seems designed only for the use of *DOI*s.
2. ***ScienceAI Crossref***: it is a *javascript* client. However, it seems very old, and is not official.
3. ***habanero***: it is not an official client, however it is maintained by something like 12 contributors. The repository is well documented, with a lot of stars. It is written in *Python*.
4. ***crossrefapi***: it is not an official client, however it is maintained by 14 contributors. The repository is not as documented as the previous one. Besides, it seems to be the choice of the community.

The *license* of *crossrefapi* is a *BSD* license. Apparently, this license is not compatible with every other licenses, which can create some conflicts with other licenses. However, it seems that it can protect developpers, giving them some legal protections.

The *license* of *habanero* is a classic *MIT* license, it is compatible with the *GNU general public license*, which is one of the most widely used open source licenses, ***this article*** says.

That's why I think I'll choose *habanero* for the *crossref API* client.

How to use the Semantic Scholar tool

There are not a lot a *SemanticScholar* clients. This one ***right here***, called *semanticscholarapi*, with around *400 stars*, is using a *MIT License*. Here is the doc for it ***right here***.

There are so many possibilities with *SemanticScholar*. It is possible to generate analytics with ***Apache Spark*** that is a processing engine, explained ***right here***.

Anyway, what I want from *Semantic Scholar* is to give me recommandations, number of citations, and keywords from a given publication.

Semantic Scholar is based on 3 different *API*s:

1. ***Datasets API*** released in *2022*. It contains *S2AG datasets* with some publications that could contain full parsed text from the publication *PDF*, paper embeddings and author attributes, ***this article*** says.

Anyway, this one is not implemented by *semanticscholarapi*.

2. ***Academic Graph API***. It can be used to retrieved some other metadata on a given publication and on authors.
3. ***Recommandations API***. This one gives recommanded papers from a simple positive example paper.

The **recommendations are very great**, it can sometimes display a list of more than 100 recommendations with all the raw data possible, including 4 different publication Ids as *DOI*, **paperId**, etc.. but not *ORCID*! (sad)

SemanticScholar is also **better at finding references and citations**. Indeed, *Crossref facets* are hard to use, here with *SemanticScholar* I just need to precise the **paperId**.

Let's talk about the **paperId**. This is wrong! Sometimes, I can't retrieve a publication because the *DOI* is not found. Indeed, the datasets (**Academic Graph** and **Recommandations API**) are using **their own paperId** to identify a research paper. And sometimes, with the right *paperId*, I can't even find the publication. It's like, buggy.

Anyway, the limit of **100 queries per 5minutes** is also tough. I think **SemanticScholar is only useful to get recommendations, and citation papers**. I will definitly use it to find recursively as many publications as possible to create the labellised dataset, but that's all, **SemanticScholar has paywalls too**.

How to use the ORCID tool

We talked about finding metadata on a publication. **It could also be interesting to find metadata on an author**

This is exactly what *ORCID* is doing, ***right here***.

There are 2 ways of using the *ORCID API*:

- "public": when u have no subscription, u will be bloqued by *paywalls*.
- "member": when u have a subscription to *ORCID*, all is accessible for you.

The big difference with other *API*s is that *ORCID* requires an authentication even for the *public* way. It is needed to get a **client-id** and a **client-secret**.

I don't want to authenticate myself. It will ask a mail and I don't know which mail provide. That's why I will use their so called ***OrcidScrapper*** that does not require any authentication. It's an alternative to *ORCID API* that just gives access to the *public* features. **OrcidScrapper can access all methods of Orcid class as it is inherited from it.**

To get the metadata about an author, it is needed to find the *orcid-id* related to the author.

I assume that *Crossref* and *SemanticScholar* could help finding this *orcid-id*. It is possible to seach using *Orcid* giving a name and a surname however it will require an *ORCID account* to make the search.

The limit for this **OrcidScrapper** *API* is:

- 12 req/sec
- 25k reads/day (IP address)

this article says.

How to use the IDREF tool

ORCID requires an *orcid-id* to find the author, which could not be found using *Crossref*. So it could be useful to use *IDREF*, which is another *API* specialized on authors, to get the *orcid-id* of the author.

IDREF for **Identifiers and Repositories for Higher Education and Research** is a public interface for consulting records produced by member of the **French higher education and research documentary networks (Sudoc, Calamec, Star)**, ***this article*** says.

For now, *IDREF* is based on *Solr*, ***this article*** says. So yeah, it seems to be quite old.

It is hard to find an *IDREF* client. This is an *API* that is not used very often.

By default, the response is in *XML*. To get *JSON*, it is needed to add `&wt=json` in the *url*.

I can't find the *request limit rate*.

Abes (=*IDREF*), has been created in *2019*, ***this article*** says:

> Pour valoriser le corpus des chercheurs, l'identifiant ORCID (Open Researcher and Contributor ID) est devenu la référence depuis plusieurs années. Depuis fin 2019, en relation avec le consortium COUPERIN, l'Abes est co-pilote du consortium ORCID France, regroupant les établissements français désireux d'adhérer aux services payants d'ORCID.

I don't want to use this *API* because there are no clients, the website of *Abes* is so old, it seems very vulnerable, and I can't find a *rate limit*. *Idref* has been created in *2019*, but the website seems to be from *2002*... Go next.

How to use the SCOPUS tool

"Scopus is the largest abstract and citations of peer-reviewing literature", ***this article*** says.

It could solve the issue of *not finding the abstract* with *Crossref*? Anyway, I just need *Scopus* to give me a correlation between an author and his *ORCID* id.

Scopus has its own *ID* for each authors because their are sometimes authors with the same name, ***this article*** says.

There is also this tool called **Scopus AI** that can generate summaries on an abstract however it is "an add-on subscription". ***this article*** says.

As other *API*'s, there are 2 ways to use it:

- *Non Commercial Users*: everything is accessible except *SciVal* and *Embase API*'s.
- *Commercial Users*: pay to win mode.

There are actually a lot of accessible data, ***this article*** says.

Unfortunately, it will be necessary to ask for an *API* key.

The documentation encourages us to use this *Python* client, called **elapsy**, ***right here***. It has a *BSD-3* License, and a lot of stars. However, it seems to be a little bit deprecated.

The *README* says to use *Pandas* on *Python 3.6*, however this example ***right here*** is not talking about it. I just hope not to get a version issue.

For the quotas, this website ***right here*** says that it depends on the service used. The reset is every seven days.

TODO

https://link.springer.com/article/10.1007/s11192-024-05073-5

EOF